

Recursion

Recursion

Recursion



Intro: Russian Dolls



You are given that each doll can only communicate with a doll 1 size bigger, or smaller than them.

Can you think of a general algorithm/behaviour for the dolls, so that they can find the total number of dolls inside them?



What is Recursion?

- A method of solving a computational problem
- Solution depends on the solutions of smaller instances of the same problem

My task to count the number of dolls inside depends on what the smaller doll says, then I can just add 1 to it, to get my answer.



Counting Dolls

$$\text{count}(n) = 1 + \text{count}(n-1)$$

... eventually

$$\text{count}(2) = 1 + \text{count}(1).$$

$$\text{Lastly, } \text{count}(1) = 0$$



Unplugged Activity

Rules:

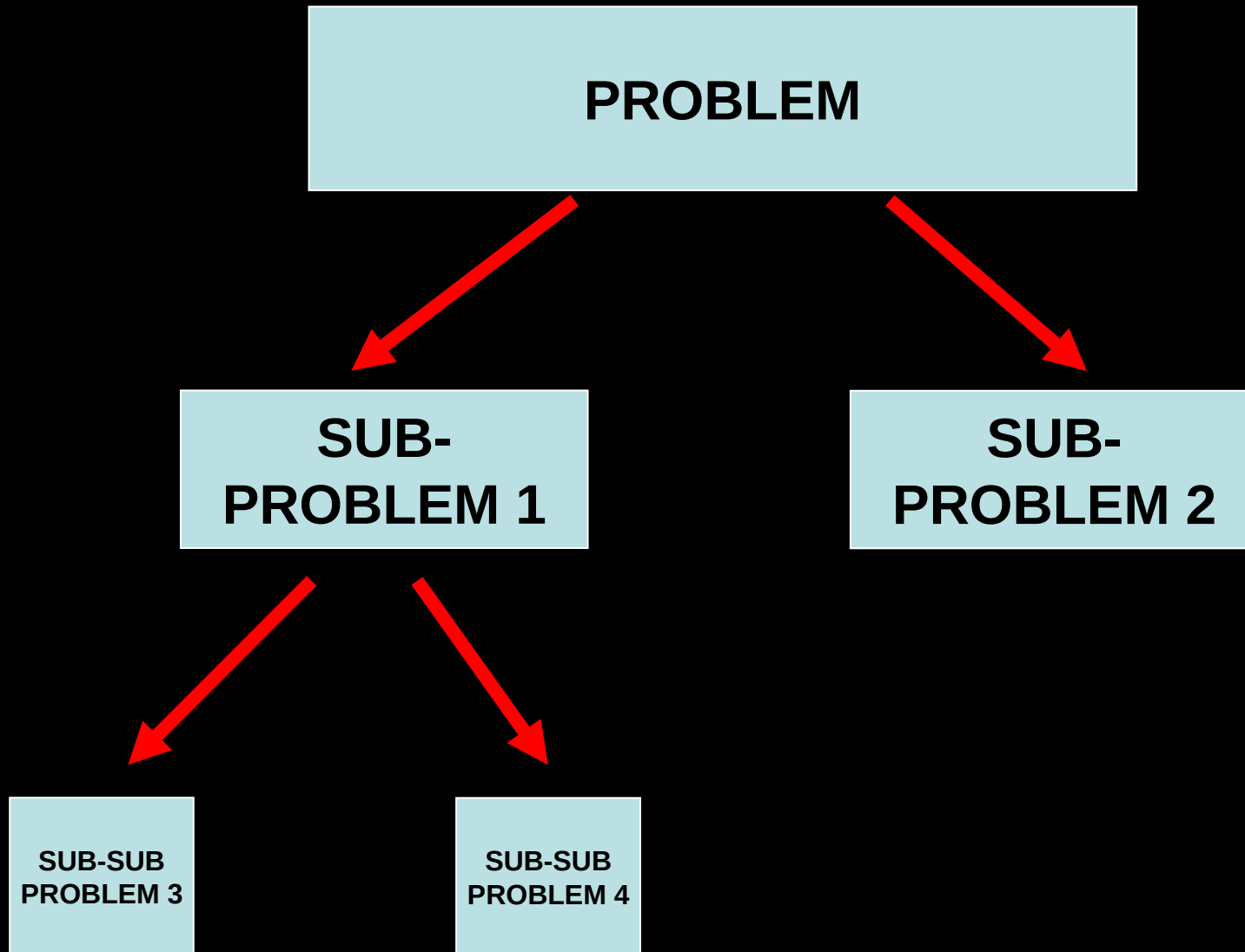
- Students are grouped together in a column.
- Students may only whisper to immediate neighbours in the column.
- Each student obtains a card with a random number. Do not show the card to anyone.

Your Task:

- Wait for further instructions.

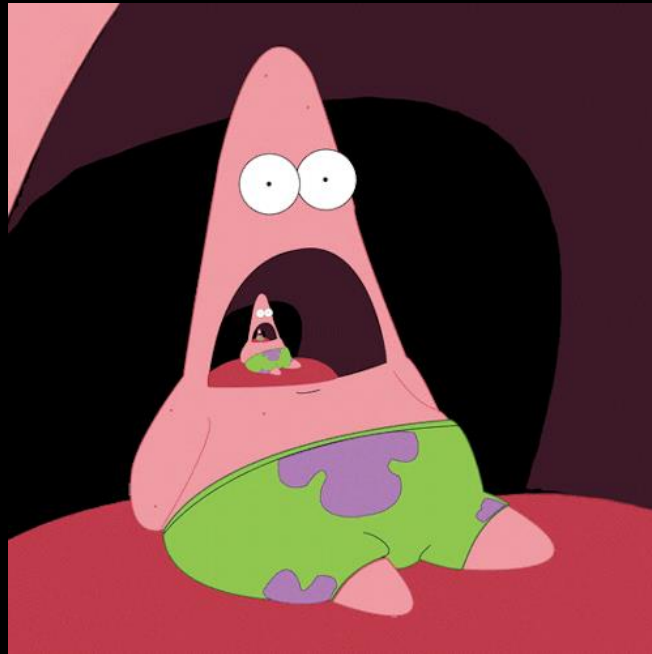


Divide-and-conquer



Criteria for Recursion

Smaller child problem(s) has
same structure as the parent



How to do recursion in python?

1. Return the answer to the simplest problem where you already know the solution
2. Use recursion to get the solution of the smaller sized problem, for e.g. $8!$ can get $7!$'s answer
3. Solve the size n problem by solving the link between the size n and the size $n - 1$ solution

My task to solve $8!$ can be done by asking my friend what is $7!$
Then I only need to multiply 8 to it, to get my answer.



Factorial Fulfills the Criteria

$$n! = n * (n-1) * (n-2) * \dots * 1$$

Same structure/pattern/relationship

$$1! = 1$$

$$2! = 2 * (1) = 2 * 1!$$

$$3! = 3 * (2 * 1) = 3 * 2!$$

$$4! = 4 * (3 * 2 * 1) = 4 * 3!$$

$$n! = n * (n - 1)!$$



Using Recursion for Factorial

The factorial function:

$$n! = n * (n-1) * (n-2) \dots * 1$$

Rewrite:

$$n! = \begin{cases} 1 & \text{if } n = 1 \\ n * (n-1)! & \text{if } n > 1 \end{cases}$$



Write the Code

$$n! = \begin{cases} 1 & \text{if } n = 1 \quad (\text{base case}) \\ n * (n-1)! & \text{if } n > 1 \quad (\text{general case}) \end{cases}$$

```
def factorial(n):  
    if n == 1: #Base Case  
        return 1  
    else:      #General Case  
        return n * factorial(n - 1)
```



Does it really work?

```
In [1]: 1 def factorial(n):  
        2     if n==1:  
        3         return 1  
        4     else:  
        5         return n * factorial(n-1)  
        6  
        7  
        8 factorial(5)
```

```
Out[1]: 120
```

Yes, it does! But how?



Tracing the recursive process using a Tree Diagram

`factorial(5)`

`= 5 * factorial(4)`

↳ `4 * factorial(3)`

↳ `3 * factorial(2)`

↳ `2 * factorial(1)`

↳ `1`

↳ `2 * 1`

↳ `3 * 2`

↳ `4 * 6`

↳ `= 5 * 24`

`= 120`

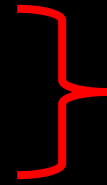
Note the chain of deferred
operations



Three Elements of a Recursive Function

```
def factorial(n):
```

```
    if n == 1:  
        return 1
```

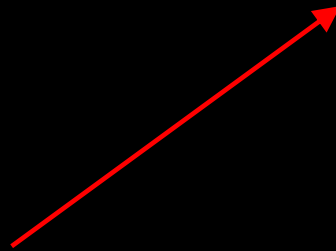


1. Base case must be present

```
else:
```

```
    return n * factorial(n - 1)
```

2. Function calls itself



3. Reducing to a smaller problem



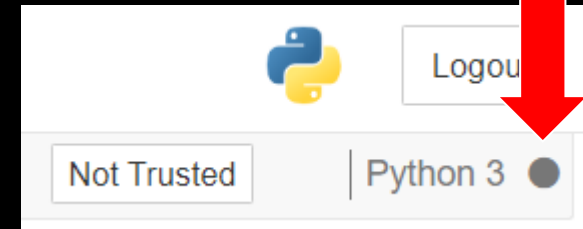
Debugging Recursion

What if your code is tested with inputs like

- `factorial(-1)`
- `factorial(2.1)`

Your code results in a maximum recursion depth error.

Kernel is dead.



```
In [2]: 1 factorial(2.1)
```

RecursionError

Traceback (most recent call last)

```
<ipython-input-2-a89507875504> in <module>()
```

```
----> 1 factorial(2.1)
```

```
<ipython-input-1-374ebd67112b> in factorial(n)
```

```
4     return 1
```

```
5     else:
```

```
----> 6     return n * factorial(n-1)
```



Debugging Recursion

```
def factorial(n):  
    print(n) ← Insert print statements on entering recursion.  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

1	factorial(4)
4	
3	
2	
1	
0	
24	

1	factorial(2.1)
2.1	
1.1	
0.100000000000000009	
-0.8999999999999999	
-1.9	
-2.9	
-3.9	
-4.9	
-5.9	

Didn't arrive
at base case.
Went past it.



Mutual recursion

```
def ping(n):  
    if n == 0:  
        return n  
    else:  
        print("Ping!")  
        pong(n - 1)  
  
def pong(n):  
    if n == 0:  
        return n  
    else:  
        print("Pong!")  
        ping(n - 1)
```

ping(7)

Ping!

Pong!

Ping!

Pong!

Ping!

Pong!

Ping!



Recursion: Summary

- 1. A recursive function has a terminating base case.**
- 2. A recursive function calls itself.**
- 3. Recursive calls reduce the problem into subproblems that eventually approach the base case.**

